When Should the Firm Take the Task Back? Error Dynamics and the Economics of Monitoring Autonomous AI Agents

Abstract

Firms increasingly delegate multistep work to autonomous AI agents while keeping the option to hand a task back to a person. We ask when that option should be exercised and what a monitoring system should track. We model delegation as an optimal stopping problem in which agent errors may excite further errors, tasks differ in difficulty, and an error may derail a task so that its output is wrong. Each channel calls for a different statistic: recency-weighted intensity under contagion, the error rate under heterogeneity, and a weighted error count under derailment, for which we derive the optimal threshold in closed form. Monitoring is most valuable for tasks at the margin of delegation. We then estimate the channels with 80,036 public software-engineering trajectories and two randomized fault-injection experiments, in customer service and code repair, with five agents and 11,352 runs. Most observed clustering reflects heterogeneity. Transient failures rarely harm recent models, whereas errors that misreport the state of the environment derail them, lowering task success by up to 30 percentage points. What an error says matters more than how many errors occur, and hand-off rules should count errors by type.

Keywords: AI agents; delegation; human–AI collaboration; monitoring; optimal stopping; Hawkes process; randomized experiment

1. Introduction

A growing share of knowledge work is now carried out by software agents built on large language models. Unlike a predictive model that returns a score for a person to act on, an agent takes a sequence of actions on its own: it queries databases, edits code, books and cancels reservations, and talks to customers. Organizations that adopt such agents rarely give up control altogether. They keep a person who can intervene, and they need a rule that tells the system when to call that person. Customer-service platforms escalate a conversation to a human representative, coding assistants stop and ask for review, and operations teams set limits on how many failed tool calls an agent may make before it is halted. These rules are information systems design choices, and at present they are set largely by intuition.

The design problem is economic. Letting the agent continue is cheap per step but risky, because an agent that has started to go wrong may keep going wrong or may finish the task with a wrong result. Taking the task back is safe but costly, because a person must pick up the work. The firm observes a stream of signals, of which tool errors are the most immediate, and must decide at each step whether the evidence justifies the cost of intervention. The question we address is therefore: when should a firm take a task back from an autonomous agent, and what should its monitoring system track to make that decision well?

The answer depends on why errors occur. Practitioners often describe agent failures as cascades: one mistake leads the agent into a confused state, and further mistakes follow. If that is how errors arise, a monitoring system should weight recent errors heavily and react quickly to bursts. Errors may instead cluster because some tasks are simply harder than others, in which case the burst is a symptom of a difficult task rather than a cause of further trouble. A third possibility, which our experiments bring to the fore, is that an error does its damage not by

producing more errors but by derailing the task, so that the agent carries on and completes the job incorrectly. These three channels, which we call contagion, heterogeneity, and derailment, have different implications for what should be monitored and when control should be withdrawn.

We develop a model of delegation in which a firm assigns a task to an agent and, at every step, chooses between letting the agent continue and handing the task to a person. Errors follow a self-exciting point process of the kind introduced by Hawkes (1971), which allows past errors to raise the rate of future ones. Task-level heterogeneity enters through an unknown baseline error rate, and derailment enters through a probability that an error spoils the eventual output. Drawing on the theory of optimal stopping (Wald 1947, Shiryaev 1963, Chow et al. 1971) and on the economics of information (Blackwell 1953, Holmström 1979), we characterize the optimal hand-off rule under each channel and under their combination.

The analysis delivers three results that matter for design. First, the sufficient statistic for the hand-off decision differs across channels. Under contagion it is a recency-weighted error intensity; under heterogeneity it is the error rate per step; under derailment it is the cumulative error count, so that a rule of the form “hand off after the N-th error” is exactly optimal, and we give N in closed form. Second, the value of contagion-based monitoring is small for agents whose error process is far from critical and grows without bound as the process approaches criticality. Third, whatever the channel, the value of monitoring is greatest for tasks at the margin of delegation, where handing the task back costs about as much as letting the agent finish. This last result gives managers a way to decide where monitoring infrastructure is worth building.

We then bring three sources of evidence to bear on the channels. The first is a corpus of 80,036 public trajectories of an open-source software-engineering agent driven by three sizes of an open-weight model. Errors in these trajectories cluster strongly, and a naive self-exciting

model implies that each error triggers more than one further error. Once task heterogeneity is modeled, the implied branching ratio falls to between 0.38 and 0.56, well below the critical value of one. Observational data cannot, however, separate contagion from persistence within a run. The second and third sources are therefore randomized experiments in two task domains, customer service and the repair of real software bugs. With five agents, including two recent commercial models, we ran 11,352 tasks and injected errors at random points, either as transient system failures or as messages that misreport the state of the environment, such as a statement that an existing reservation cannot be found. Transient failures rarely raised later error rates and left the success of the commercial models unchanged. Misleading errors produced short bursts of further errors and, more importantly, derailed the task: in customer service they lowered the success of GPT-6.1-terra by 30 and of Claude Sonnet 5.5 by 15 percentage points. The evidence points away from cascades and toward derailment, and it shows that the derailment probability depends on what the error says, on the agent, and on the task.

The paper makes three contributions. For research on human–AI delegation in information systems (Baird and Maruping 2021, Fügener et al. 2022), it offers a formal account of when control should be withdrawn from an agentic system and shows that the answer turns on an identifiable property of the agent's error process. For research on IS control (Kirsch 1997), it connects the choice of what to monitor to the informativeness principle: a monitoring signal is worth collecting only to the extent that it is informative about the state that drives costs. For practice, it shows that count-based escalation rules, which are already common, are optimal under the channel that our experiments find to dominate, provided that errors are counted by type, and it shows how the weights and the threshold can be calibrated from a modest experiment of the kind we report.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 presents the model, and Section 4 derives the results. Section 5 reports the observational and experimental evidence and calibrates the model. Section 6 discusses implications and limitations, and Section 7 concludes. Proofs and experimental details are in the appendices.

2. Related Literature

2.1. Delegation to Agentic Systems

Information systems research has long examined how people allocate work between themselves and technology. Recent work treats AI systems as agents to which tasks are delegated rather than as tools that people use. Baird and Maruping (2021) develop a framework for delegation to and from agentic IS artifacts and emphasize that delegation involves an ongoing allocation of rights and responsibilities. Berente et al. (2021) describe the management of AI as a problem of balancing autonomy, learning, and inscrutability. Fügener et al. (2022) show experimentally that people delegate poorly to AI because they misjudge their own and the system's competence, which suggests that delegation decisions are better made by rule than by unaided judgment. Our work shares this focus on delegation but addresses a question that becomes central once agents act over many steps: not whether to delegate, but when to revoke a delegation already made.

2.2. Control and Monitoring in Information Systems

A second stream studies control, the means by which a principal ensures that work is carried out as intended. Kirsch (1997) shows that managers of IS projects combine formal and informal control modes and adapt them to what can be observed and measured. Monitoring an

autonomous agent is a formal behavior control in this sense, and the question of which behaviors to measure is the familiar one of choosing control mechanisms that are informative about outcomes. In agency theory, Holmström (1979) shows that a signal should enter a contract if and only if it carries information about the agent's action beyond what other signals already provide. We apply the same logic to a stopping problem: the timing of errors should enter a hand-off rule only when it is informative about the hidden state that determines future costs.

2.3. Optimal Stopping, Learning, and the Value of Information

The decision to take a task back is a stopping problem. Sequential analysis (Wald 1947) and quickest detection (Shiryaev 1963) study when to stop observing a process and act, and the theory of monotone stopping problems (Chow et al. 1971) identifies conditions under which a myopic rule is optimal; we use that result to obtain a closed-form threshold under derailment. When the agent's error rate on a task is unknown, the firm learns about it as the task unfolds, much as a worker learns about the quality of a job match and quits when the posterior turns unfavorable (Jovanovic 1979). Blackwell (1953) provides the general notion of when one information structure is more valuable than another, and our result on the value of monitoring at the margin of delegation is a specific instance of the principle that information is valuable only when it can change a decision.

2.4. Heterogeneity, State Dependence, and Agent Reliability

Distinguishing true contagion from heterogeneity is a classic problem in econometrics. Heckman (1981) shows that persistence in observed outcomes can arise either because past outcomes change future behavior or because units differ in unobserved ways, and that the two have different policy implications. The same issue arises for agent errors, and we address it with a random-effects self-exciting model and, more decisively, with randomized injection. In

computer science, benchmarks such as SWE-agent (Yang et al. 2024) and τ-bench (Yao et al. 2024, Barres et al. 2025) measure whether agents complete tasks, and a growing literature documents how agents fail. These studies report success rates; they do not model the dynamics of errors within a run or derive the decision rules that a firm deploying agents needs.

3. Model

A firm assigns a task to an AI agent. Time is discrete and indexed by steps t = 0, 1, 2, …, where a step is one action of the agent, such as a tool call. At the start of each step the firm either lets the agent take the step or hands the task to a person. Handing off ends the agent's involvement at cost $H > 0$, which includes the person's time to take over and complete the task. A step costs $k > 0$ plus $c > 0$ for each error made in it. After each step the agent completes the task with probability $q \in (0, 1)$.

Errors follow a discrete-time self-exciting process. The number of errors in step t is

$$e_t \mid x_t \sim Poisson(\mu + x_t), \quad x_{t+1} = \delta x_t + \alpha e_t, \quad x_0 = 0, \tag{1}$$

where $\mu > 0$ is the baseline error rate, $x_t \geq 0$ is the excess intensity built up by past errors, $\alpha \geq 0$ is the increase in intensity caused by one error, and $\delta \in [0, 1]$ is the rate at which that increase persists from one step to the next. Equivalently, $x_t = \alpha \Sigma_{s<t} \delta^{t-1-s} e_s$. The branching ratio $n = \alpha/(1 - \delta)$ is the expected number of further errors that a single error triggers, directly or through its descendants. Because $E[x_{t+1} \mid x_t] = \rho x_t + \alpha\mu$ with $\rho = \delta + \alpha$, the error process is stable if and only if $n < 1$; we call an agent subcritical, critical, or supercritical according to whether n is below, equal to, or above one.

We allow two further channels. Heterogeneity: the baseline rate μ is specific to the task and drawn from a gamma distribution with shape a and rate b; the firm knows the distribution but not the draw. Derailment: each error independently derails the task with probability $\theta \in [0,$

1]. A derailed agent continues to work and still completes the task with probability q per step, but its output is wrong, and the firm then bears an additional cost $F > 0$ to detect and repair it. Derailment is not observed directly; the firm sees only errors. The parameter θ captures the experimental finding, reported in Section 5, that errors lower the probability of a correct outcome without necessarily producing further errors.

We maintain the following assumptions throughout. (A1) The firm observes the errors made in each step. (A2) The parameters (μ or its distribution, α, δ, θ, k, c, q, H, F) are known and constant over time. (A3) Completion occurs with probability q per step regardless of the error history; errors affect whether the completed output is correct only through derailment. Assumption A3 replaces the stronger assumption, common in reliability models, that errors leave the outcome unaffected. We analyze each channel in isolation before turning to their combination. With μ known and $\theta = 0$, the state is the excess intensity x, and the minimum expected cost W satisfies

$$W(x) = \min\{H, C(x)\}, \quad C(x) = k + c(\mu + x) + (1 - q)\, E[W(\delta x + \alpha e)], \quad e \sim \text{Poisson}(\mu + x). \tag{2}$$

4. Analysis

4.1. Contagion: A Threshold on Excess Intensity

Proposition 1. *Equation (2) has a unique bounded solution W, which is nondecreasing in x. The firm hands the task back if and only if $x_t \geq x^*$, where $x^* = \inf\{x : C(x) \geq H\}$.*

The intuition is straightforward. A higher excess intensity raises the expected error cost of the current step and, because the Poisson family is ordered by first-order stochastic dominance, makes the next state worse as well. The proof in Appendix A uses the fact that the Bellman operator is a contraction with modulus $1 - q$ and preserves monotonicity. In numerical

work the threshold falls as the branching ratio rises, from 2.17 at n = 0 to 0.67 at n = 1.05 under the calibration used in Section 5: more contagious agents should be taken back sooner.

Proposition 2. *If $\delta = 1$, the rule that hands off when the cumulative error count N_t reaches a fixed bound is optimal. If $\delta < 1$, $\alpha > 0$, and x^* lies in the interior of the attainable range, every rule based on N_t alone is strictly suboptimal.*

Proposition 2 is the informativeness principle applied to a stopping problem. What determines future costs is the excess intensity, and the error count is a sufficient statistic for it only when the effect of an error never decays. When it does decay, two histories with the same number of errors but different timing imply different intensities, and a rule that ignores timing must treat them alike. A monitoring system for a contagious agent should therefore weight recent errors more heavily than old ones.

Proposition 3. *If the firm never hands off and $\theta = 0$, its expected cost is $C_U = (k + c\mu)/q + [c\alpha\mu/(1 - \rho)]\cdot[1/q - 1/(1 - (1 - q)\rho)]$, which is finite if and only if $(1 - q)\rho < 1$.*

Proposition 4. *Let $V_C = \min\{H, C_U\} - W(0)$ be the value of contagion-based monitoring relative to the better of the two static choices (hand off immediately or never). Then $V_C \geq 0$, $V_C = 0$ when $\alpha = 0$, and $C_U - W(0)$ grows without bound as $(1 - q)\rho$ approaches one.*

Propositions 3 and 4 locate the value of contagion-based monitoring near criticality. For an agent well below criticality, an error raises the intensity briefly, and the threshold in Proposition 1 is rarely reached; the optimal rule then differs little from never intervening. In the calibration of Section 5 the value relative to never handing off is zero up to n = 0.4 and rises to 4.5 and 30.7 units of step cost at n = 0.8 and n = 1.0.

4.2. Heterogeneity: Monitoring as Learning

Suppose next that $\alpha = \theta = 0$ but that μ is unknown. After t steps with N_t errors the posterior is gamma with shape $a + N_t$ and rate $b + t$, and errors in the next step follow a negative binomial distribution with mean $(a + N_t)/(b + t)$. The state is (t, N_t).

Proposition 5. *Under heterogeneity alone, the firm hands off at step t if and only if $N_t \geq N^*(t)$. As t grows, the rule approaches a rate rule: hand off when $(a + N_t)/(b + t)$ exceeds $\mu^* = (qH - k)/c$.*

The count remains sufficient here, but its bound must rise with the number of steps taken. Five errors in five steps signal a hard task; five errors in eighty do not. This is the logic of quitting a poor job match. Propositions 2 and 5 give opposite prescriptions, one favoring recency and the other normalizing by elapsed time, and the econometric question of which channel dominates therefore has direct design consequences.

4.3. Derailment: The Error Count Is Sufficient

Now let $\alpha = 0$ and μ be known, but let each error derail the task with probability $\theta > 0$. After N errors the posterior probability that the task is derailed is $\pi_N = 1 - (1 - \theta)^N$, which depends on the history only through the count. A step costs $k + c\mu$ in expectation; if the task completes, the firm bears F with the probability that it has been derailed by then.

Proposition 6. *Under derailment, the optimal policy hands the task back as soon as the cumulative error count reaches N^*, the smallest integer N such that*

$$(1 - \theta)^N \leq [1 - (qH - k - c\mu)/(qF)] \cdot e^{\mu\theta}, \tag{3}$$

with $N^* = 0$ if $qH \leq k + c\mu$ (the agent is never worth using) and $N^* = \infty$ if the right-hand side is not positive (the agent is always worth using).

The proof (Appendix A) uses the monotone case of optimal stopping. Because the count never decreases and the one-step gain from continuing falls as the count rises, the set of states in which stopping beats one more step is absorbing, and the myopic rule is optimal (Chow et al. 1971). The proposition gives a direct justification for count-based escalation rules of the kind already used in practice and shows how to set the bound. The threshold falls as θ and F rise and as H falls. It does not depend on when the errors occurred, which is the opposite of the conclusion under contagion. Value iteration on the count reproduces the closed form in every parameter configuration we examined.

Errors differ in kind, and our experiments show that different kinds derail a task with different probabilities. Suppose that errors of type k = 1, …, K arrive at rates μ_k and derail the task with probabilities θ_k.

Corollary 1. *With several error types, the optimal policy hands the task back as soon as the weighted count Σ_k w_k N_k reaches a threshold, where N_k is the number of errors of type k and w_k = −ln(1 − θ_k). The threshold is given by Equation (3) with ln(1 − θ)·N replaced by −Σ_k w_k N_k and μθ replaced by Σ_k μ_k θ_k.*

The corollary follows because the probability that the task is still on track is Π_k (1 − θ_k)^{N_k}, so the log of that probability is linear in the counts. An error type that never derails a task receives a weight of zero and should not be counted at all, however often it occurs.

4.4. Combining the Channels

Proposition 7. *When contagion and derailment operate together, the state is (x, N). The minimum expected cost is nondecreasing in both arguments, the stopping region is an upper set, and the critical count N*(x) is nonincreasing in x.*

When both channels are present, the optimal rule is a count threshold that is lowered temporarily after a burst of errors. How much the burst matters depends on δ. When excitation decays within a few steps, as our experiments suggest, the adjustment applies only to the steps immediately after an error, and the rule is close to a pure count rule.

4.5. Where Monitoring Is Worth Building

Proposition 8. *For each channel and for their combination, let $\bar{C}$ be the expected cost of never handing off and let $V(H) = \min\{H, \bar{C}\} - W(0; H)$ be the value of monitoring as a function of the hand-off cost. Then V is nondecreasing on $[0, \bar{C}]$, nonincreasing on $[\bar{C}, \infty)$, and maximized at $H = \bar{C}$.*

The proof is short. Every policy has an expected cost of the form $A + H \cdot P$, where P is the probability that it hands off, so the optimal cost is a lower envelope of affine functions of H and is therefore concave. The value of monitoring is then convex below $\bar{C}$ and nonincreasing above it. The result says that monitoring pays most for tasks at the margin of delegation, where taking the task back costs about as much as letting the agent finish. Tasks that are cheap to reassign can simply be reassigned, and tasks that are very expensive to reassign will be left with the agent whatever the monitor reports. Figure 1 shows the value of contagion-based and learning-based monitoring as functions of H; both peak exactly at the corresponding $\bar{C}$.

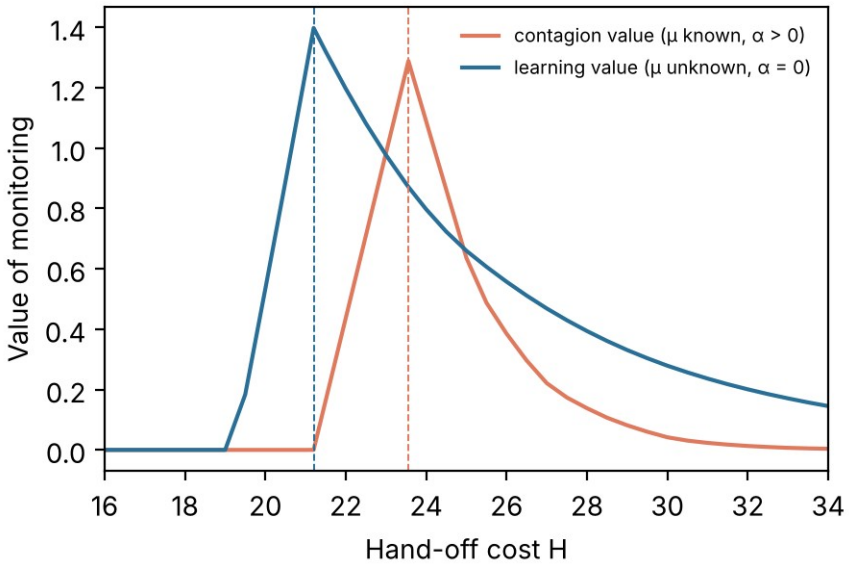


Figure 1. Value of monitoring as a function of the hand-off cost H. The dashed lines mark the expected cost of letting the agent finish (23.56 under contagion, 21.20 under heterogeneity). Parameters: k = 1, c = 5, q = 0.06, with error parameters from the Llama-3.1-70B estimates in Table 1.

5. Empirical Evidence

The theory makes the design of a monitoring system depend on three quantities: the strength and persistence of contagion (α and δ), the dispersion of task difficulty, and the probability that an error derails a task (θ). We estimate them in two steps. Observational data on a large number of agent runs tell us how much errors cluster and how much of the clustering can be attributed to heterogeneity. Two randomized experiments, in different task domains, then identify the causal effect of an error on later errors and on task success.

5.1. Observational Evidence from Software-Engineering Trajectories

We use a public corpus of trajectories produced by SWE-agent (Yang et al. 2024), an open-source agent that resolves GitHub issues, driven by three sizes of Llama-3.1 (Nebius 2024). The corpus contains 80,036 trajectories. We code an error whenever the environment rejects an agent action: an edit that fails a syntax check, a command that does not exist, a missing file, a

malformed call, or a timeout. Outputs of the file viewer are excluded because they cannot fail in this sense. About 22% of steps contain an error.

Errors cluster strongly. The probability of an error is 0.66 after an error and 0.10 after a successful step. Part of this gap reflects heterogeneity: when errors are permuted within each trajectory, which preserves each trajectory's error rate and destroys any dependence, the gap is 0.434 on average, against an observed 0.555. A conditional logit model with trajectory fixed effects gives a positive coefficient on decayed past errors (0.25 to 0.31 across decay rates) where the same model on permuted sequences gives -0.33.

Table 1 reports maximum likelihood estimates of the self-exciting model with and without a gamma-distributed baseline. Without heterogeneity, the branching ratio exceeds one for all three models, which would place them in the supercritical regime where Proposition 4 makes monitoring very valuable. With heterogeneity, the ratio falls by a factor of three to four, to between 0.38 and 0.56, and the likelihood ratio tests reject both restricted models. Two features of the data caution against reading the remaining excitation as causal. Persistence within a run, such as an agent stuck in a malformed workspace, produces the same pattern as contagion. And the early-warning value of the error stream is small: the number of errors in the first eight steps predicts eventual failure with an area under the curve of only 0.55.

Table 1. Self-Exciting Models of Agent Errors in SWE-Agent Trajectories

Model	Trajectories	n, no heterogeneity	n, with heterogeneity	95% bootstrap	α	δ	CV of μ	LR, excitation
Llama-3.1-8B	3,000	1.85	0.50	[0.44, 0.56]	0.20	0.60	1.79	2077
Llama-3.1-70B	3,000	1.48	0.38	[0.31, 0.42]	0.21	0.45	1.41	1130
Llama-3.1-405B	1,140	2.08	0.56	[0.49, 0.62]	0.15	0.73	3.05	659

Notes. Discrete-time Hawkes model with baseline $\mu_i(1 + t)^{\gamma}$ and gamma-distributed μ_i, first 60 steps of each trajectory with at least six steps. Up to 3,000 trajectories per model. $n = \alpha/(1 - \delta)$. The likelihood ratio (LR) tests excitation given heterogeneity (two degrees of freedom). Bootstrap intervals use 15 resamples warm-started at the estimate and may understate uncertainty. Estimates in the subcritical range are robust to five starting values.

5.2. Two Randomized Fault-Injection Experiments

Design common to both experiments.

To separate contagion from persistence, we injected errors at random into agent runs. We ran five agents: Llama-3.1-8B and Llama-3.1-70B, which match the observational data; DeepSeek-V3.1, a stronger open-weight model; and two recent commercial models, GPT-6.1-terra (OpenAI) and Claude Sonnet 5.5 (Anthropic), both with low reasoning effort. Each task was run in four arms. In the control arm nothing was changed. In the transient arm the agent's j-th tool call, with j drawn uniformly from {1, 2, 3}, was not executed and returned a message stating that the service was temporarily unavailable. In the misleading arm the call returned instead a message of the form "<argument> '<value>' not found," which mimics the environment's genuine error messages; it reads like the agent's own mistake and asserts, falsely, that the requested object does not exist. In the repeated arm each call after the first was replaced by the unavailability message with probability 0.25, up to three times. Control runs carry a virtual injection point drawn in the same way. Injected calls were never executed, so they could not change the state of the environment.

We measure agent errors as tool calls that the real environment rejects, excluding the injected calls themselves. Because the injected errors are randomized, a comparison of later error rates between injected and control runs identifies the causal effect of an error on subsequent errors, the quantity α in the model, and the comparison of success rates identifies the total effect of an error on the outcome, which maps into θ.

Study 1: customer service.

We used τ²-bench (Barres et al. 2025), a public benchmark in which an agent serves simulated customers of an airline (50 tasks) and a retailer (114 tasks) through tool calls on a database. Success is verified by comparing the final database with the one produced by the correct actions. The simulated customer was played by GPT-4.1-mini. Each task–arm cell was run three times for the Llama models and twice for the other agents, giving 7,872 runs.

Study 2: code repair.

We used 87 real bugs from seven open-source Python projects in BugsInPy (Widyasari et al. 2020), selected because the developer's test fails on the buggy version and passes after the developer's fix in our environment. The agent receives the failing test and its output and can list directories, view files, search, edit a file by replacing a unique string, run the failing test, and submit, for at most 30 turns. The environment rejects calls on missing paths, edits whose target string is absent or ambiguous, edits that would break Python syntax, attempts to edit test files, and malformed calls. Success requires that every failing developer test passes and that no test that passed before fails. Each bug–arm cell was run twice, giving 3,480 runs.

5.3. Experimental Results

Table 2. Task Success by Experimental Arm

Domain	Agent	Control success	Transient − control	Misleading − control	Repeated − control
Customer service	Llama-3.1-8B	0.18	+0.016 [-0.012, 0.041]	-0.022 [-0.047, 0.002]	-0.010 [-0.030, 0.012]
Customer service	Llama-3.1-70B	0.33	-0.039 [-0.085, 0.006]	-0.043 [-0.087, 0.006]	-0.093 [-0.140, -0.047]
Customer service	DeepSeek-V3.1	0.68	-0.198 [-0.259, -0.143]	-0.195 [-0.262, -0.131]	-0.116 [-0.177, -0.058]
Customer service	GPT-6.1-terra	0.77	-0.006 [-0.061, 0.049]	-0.305 [-0.372, -0.235]	-0.037 [-0.088, 0.018]
Customer service	Claude Sonnet 5.5	0.84	-0.021 [-0.070, 0.021]	-0.146 [-0.210, -0.088]	+0.003 [-0.037, 0.040]
Code repair	Llama-3.1-8B	0.00	+0.011 [0.000, 0.034]	+0.011 [0.000, 0.029]	+0.006 [0.000, 0.017]

Code repair	Llama-3.1-70B	0.09	+0.034 [-0.011, 0.086]	-0.006 [-0.063, 0.046]	+0.006 [-0.035, 0.052]
Code repair	DeepSeek-V3.1	0.83	-0.080 [-0.144, -0.017]	-0.057 [-0.115, -0.006]	-0.063 [-0.109, -0.011]
Code repair	GPT-6.1-terra	0.99	+0.006 [0.000, 0.017]	+0.006 [0.000, 0.017]	+0.006 [0.000, 0.017]
Code repair	Claude Sonnet 5.5	0.97	-0.052 [-0.098, -0.011]	-0.057 [-0.109, -0.011]	-0.034 [-0.075, 0.000]

Notes. Differences are paired by task (or bug) and repetition; 95% intervals from 1,000 bootstrap resamples of tasks.

Table 3. Effect of Errors on Subsequent Agent Errors

Domain	Agent	Base error rate	Transient, lag 1	Misleading, lag 1	Misleading, lag 2	Own error, lag 1	Δ 5 calls, transient	Δ 5 calls, misleading
Customer service	Llama-3.1-8B	0.654	-0.34 (0.08)	+0.42 (0.12)	+0.01 (0.12)	+0.66 (0.04)	-0.037	-0.022 [-0.056, 0.010]
Customer service	Llama-3.1-70B	0.190	-0.46 (0.12)	-0.56 (0.15)	-0.48 (0.16)	+0.19 (0.06)	+0.007	-0.009 [-0.034, 0.016]
Customer service	DeepSeek-V3.1	0.055	-0.44 (0.24)	+0.90 (0.20)	-0.35 (0.33)	+0.59 (0.14)	+0.000	+0.038 [0.018, 0.058]
Customer service	GPT-6.1-terra	0.012	+2.14 (0.66)	+4.50 (0.60)	+3.23 (0.83)	+0.45 (0.42)	+0.006	+0.021 [0.011, 0.035]
Customer service	Claude Sonnet 5.5	0.014	+0.43 (0.59)	+3.06 (0.40)	+2.75 (0.46)	+0.31 (0.32)	+0.002	+0.026 [0.014, 0.040]
Code repair	Llama-3.1-8B	0.812	-0.65 (0.14)	+0.53 (0.24)	+0.90 (0.24)	+0.82 (0.09)	-0.121	-0.061 [-0.112, -0.015]
Code repair	Llama-3.1-70B	0.429	+0.03 (0.11)	-0.18 (0.20)	-0.27 (0.21)	+1.07 (0.06)	+0.014	-0.053 [-0.119, 0.011]
Code repair	DeepSeek-V3.1	0.029	-0.30 (0.33)	+3.31 (0.22)	+1.94 (0.28)	+0.13 (0.21)	+0.006	+0.091 [0.070, 0.115]
Code repair	GPT-6.1-terra	0.021	+0.34 (0.42)	+2.51 (0.30)	+2.56 (0.34)	-0.41 (0.46)	+0.002	+0.039 [0.025, 0.054]
Code repair	Claude Sonnet 5.5	0.012	-1.19 (1.03)	+0.18 (0.77)	-0.63 (1.05)	+0.57 (0.49)	-0.001	+0.006 [-0.005, 0.019]

Notes. Columns 4–7: logit of an agent error on indicators of injected and own errors one or two calls earlier, with task fixed effects and the log position of the call, estimated on tasks with variation in the outcome; coefficients (standard errors) are relative effects. n.i.: not identified because almost no errors follow this event. Columns 8–9: difference in the agent error rate over the five calls after the (virtual) injection point, an absolute effect; 95% interval from a task-clustered bootstrap.

Four findings stand out (Tables 2 and 3, Figures 2 and 3). First, transient failures rarely caused further errors. After an injected unavailability message, the log-odds of an error on the next call fell or did not change in nine of the ten agent–domain cells. The exception is GPT-6.1-terra in customer service (+2.14), where the base rate is about 1% and the five-call error rate rose by 0.6 percentage points. The agents typically retried, and the retry usually succeeded.

Second, misleading errors produced bursts of further errors in the more capable agents. The lag-one coefficients range from +2.5 to +4.5 for DeepSeek-V3.1 and GPT-6.1-terra in code repair and for both commercial models in customer service, and the effect persists for two or three calls. In absolute terms the bursts are small, because these agents rarely err: the error rate over the five calls after the injection rose by between 2 and 9 percentage points. A plausible mechanism is that the agent, told that an object it needs does not exist, searches for alternatives that do not exist either. Third, for the weaker agents the coefficients on their own past errors are larger than the causal coefficients on injected errors, so most of the clustering seen in uncontrolled data reflects persistence of difficulty within a run, consistent with the heterogeneity-adjusted estimates in Table 1.

Fourth, and most consequential, errors derailed tasks, and how much they did so depended on what the error said, on the agent, and on the domain. In customer service, a single misleading error lowered the success of GPT-6.1-terra from 0.77 to 0.46 and that of Claude Sonnet 5.5 from 0.84 to 0.69, whereas a transient failure left both unchanged. DeepSeek-V3.1 lost 20 points after either kind of error. Because these agents made almost no further errors, the loss must come from what they did after the injected message rather than from further failed calls; a plausible mechanism, which we did not code systematically, is that the agent accepts the false report and acts on it, for example by telling the customer that the reservation does not exist.

In code repair the effects were smaller: DeepSeek-V3.1 and Claude Sonnet 5.5 lost between 5 and 8 points, and GPT-6.1-terra, which repaired 99% of the bugs, was unaffected. One reason may be that a coding agent can run the tests and discover that something is wrong, whereas a customer-service agent receives no comparable feedback. The two Llama models succeeded rarely in either domain and had little to lose.

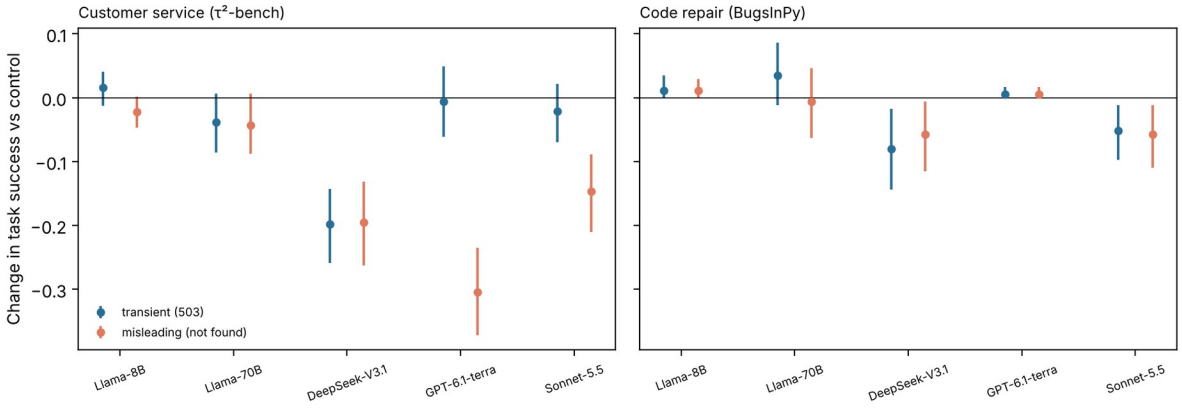


Figure 2. Change in task success relative to the control arm after one injected transient (503) or misleading (not found) error, with 95% intervals, by agent and domain.

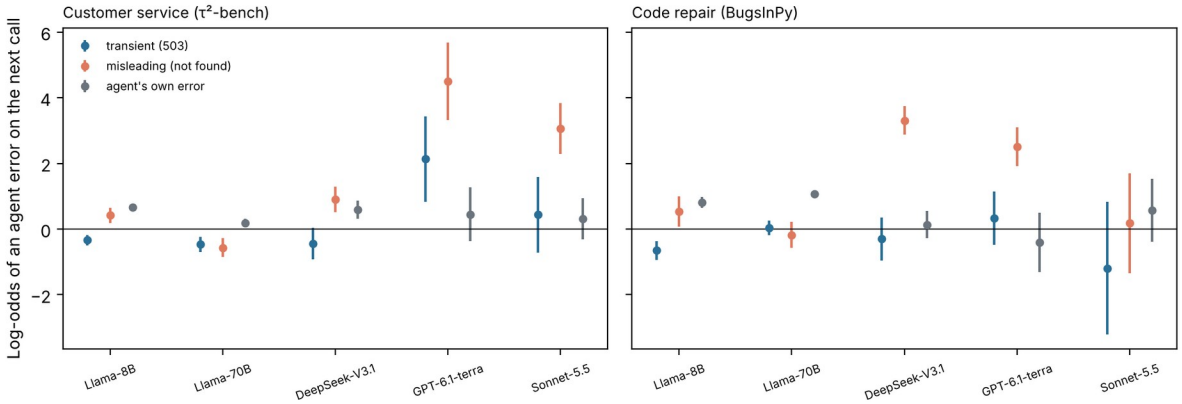


Figure 3. Logit coefficients of an agent error on the next call after an injected transient error, an injected misleading error, and the agent's own error, with 95% intervals. Injected errors are randomized. Coefficients that are not identified are omitted.

5.4. Calibrating the Hand-Off Rule

The experiments allow us to estimate θ directly. If an error derails a task with probability θ and derailment turns a success into a failure, the ratio of success rates in the injected and control arms is 1 − θ. Table 4 reports θ for each agent, domain, and kind of error, together with the thresholds implied by Proposition 6 for two values of the cost of a wrong output F, holding the other costs at the values used above (k = 1, c = 5, q = 0.06, H = 40, μ = 0.054). Where θ cannot be distinguished from zero the rule recommends no count-based intervention for that kind of error.

Table 4. Derailment Probability and Optimal Hand-Off Thresholds

Domain	Agent	Control success	θ, transient	θ, misleading	N*, transient (F = 60 / 100)	N*, misleading (F = 60 / 100)
Customer service	Llama-3.1-8B	0.18	-0.09 [-0.27, 0.05]	0.13 [-0.03, 0.25]	none / none	none / none
Customer service	Llama-3.1-70B	0.33	0.12 [-0.03, 0.25]	0.13 [0.00, 0.25]	none / none	none / none
Customer service	DeepSeek-V3.1	0.68	0.29 [0.21, 0.37]	0.29 [0.20, 0.37]	2 / 1	2 / 1
Customer service	GPT-6.1-terra	0.77	0.01 [-0.06, 0.08]	0.40 [0.31, 0.48]	none / none	1 / 1
Customer service	Claude Sonnet 5.5	0.84	0.03 [-0.03, 0.08]	0.18 [0.11, 0.24]	none / none	2 / 2
Code repair	Llama-3.1-8B	0.00	—	—	—	—
Code repair	Llama-3.1-70B	0.09	-0.40 [-1.50, 0.12]	0.07 [-0.85, 0.57]	none / none	none / none
Code repair	DeepSeek-V3.1	0.83	0.10 [0.02, 0.17]	0.07 [0.01, 0.14]	4 / 2	6 / 3
Code repair	GPT-6.1-terra	0.99	-0.01 [-0.02, 0.00]	-0.01 [-0.02, 0.00]	none / none	none / none
Code repair	Claude Sonnet 5.5	0.97	0.05 [0.01, 0.10]	0.06 [0.01, 0.11]	7 / 4	7 / 4

Notes. θ = 1 − success(injected)/success(control), with 95% task-clustered bootstrap intervals. N* from Proposition 6; “none” where the interval for θ includes zero. Dashes mark agents whose control success is below 5%, for which θ is not estimable. Value iteration reproduces every closed-form threshold.

The calibrated rules differ sharply by kind of error. For GPT-6.1-terra in customer service, a single misleading error should prompt a hand-off when F = 60, whereas transient

failures should not be counted at all; by Corollary 1, an escalation rule that adds up errors of both kinds would be badly miscalibrated. For DeepSeek-V3.1 in customer service, two errors of either kind suffice. In code repair the thresholds are higher, between four and seven errors at F = 60, reflecting the smaller derailment probabilities in that domain. Figure 4 shows how the value of the rule rises with θ and with the cost of a wrong output: monitoring is most valuable for capable agents deployed on tasks where an undetected wrong output is expensive and the agent cannot check its own work.

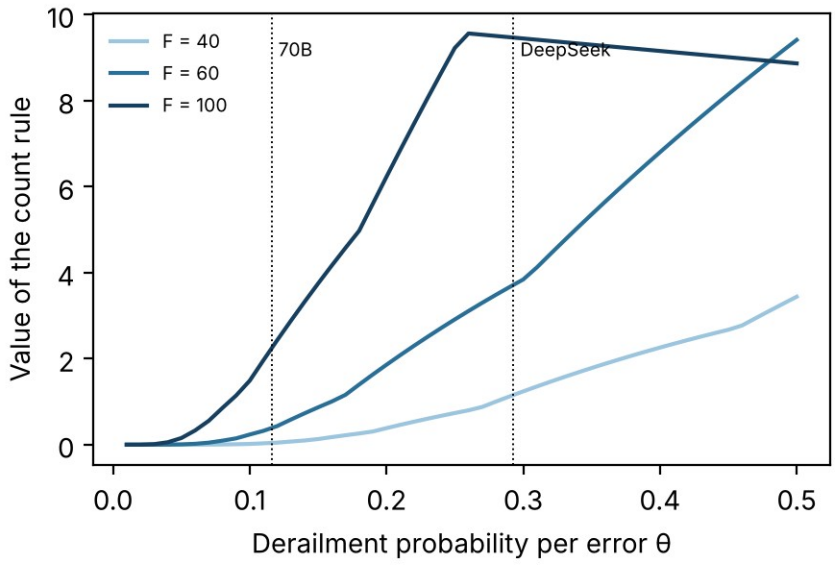


Figure 4. Value of the optimal count rule relative to the better static choice, as a function of the derailment probability θ, for three costs of a wrong output F. Dotted lines mark the customer-service estimates for Llama-3.1-70B and DeepSeek-V3.1.

6. Discussion

6.1. Implications for Research

The central implication is that the design of human oversight for agentic systems should start from the mechanism that generates agent errors. The same observational pattern, bursts of errors, is consistent with contagion, heterogeneity, and persistent difficulty, and these

mechanisms call for different signals. Research on human–AI delegation has concentrated on whether and how much people should rely on AI (Fügener et al. 2022). Once agents act over many steps, the analogous question is dynamic: under what evidence should a delegation be revoked? Our model shows that the answer can be stated in terms of a few estimable parameters, and our experiments show that these parameters can be measured with a design that any firm deploying an agent could replicate.

The results also qualify an influential intuition about agent failure. The image of an agent that spirals after its first mistake is supported by the raw data, which show strong clustering, but only partly by the randomized evidence. Transient failures leave later error rates largely unchanged, and misleading errors raise them for a few calls, by a few percentage points. What the evidence supports more strongly is that errors change outcomes, and that the change depends on what the error says. A theory of agent reliability that counts errors without asking what they convey will misjudge risk.

This last point connects agent oversight to a familiar concern of the IS field, the quality of the information that systems provide (Wang and Strong 1996). The commercial models in our experiments shrugged off a service outage but acted on a false statement that a record did not exist, and they did so more readily than the weaker open models. Capable agents appear to trust their tools. As firms connect agents to enterprise data, the accuracy of tool outputs therefore becomes a determinant of agent performance, and data governance becomes part of AI governance. For research on control, the derailment channel also suggests that outcome controls, such as verification of the final result, deserve attention alongside the behavior controls that monitoring represents.

Finally, the analysis illustrates the value of combining econometric decomposition with experimentation in research on AI systems. Agent behavior can be manipulated at low cost and with full control over the treatment, which makes experiments of the kind we report inexpensive; the 11,352 runs reported here cost about US$330 in model usage. This opens a path to testing behavioral claims about AI agents with the same standards that the field applies to claims about people.

6.2. Implications for Practice

Three design principles follow. First, before building elaborate monitoring, a firm should classify the errors its agent encounters and establish how each kind affects outcomes. In our experiments transient failures were nearly harmless for recent models, whereas errors that misreported the environment were not; by Corollary 1, the former should carry little or no weight in an escalation rule and the latter a large one. Second, the weights and the threshold should be set from the derailment probabilities and the cost of a wrong output, which can be estimated with a small injection experiment on the firm's own tasks. The estimates must be domain specific: the same model was derailed far more often in customer service, where it relayed a false statement to the customer, than in code repair, where running the tests exposed the problem. Third, monitoring investment should be targeted at tasks near the margin of delegation, where the cost of taking the task back is close to the expected cost of letting the agent finish (Proposition 8).

6.3. Limitations and Future Research

Several limitations qualify our conclusions. The experiments cover two task domains, customer service with a simulated customer and the repair of bugs in seven open-source Python projects, and two kinds of injected error. The misleading message reads like the agent's own mistake and at the same time asserts something false about the environment; our design does not

separate these two features, and a message that is accurate but blames the agent might act differently. The injected errors are exogenous by design and may differ from the agent's own errors, which arise when the agent is already confused. Model versions change quickly, the commercial models were run with low reasoning effort, and requests were served by more than one provider. Llama-3.1-8B frequently produced malformed tool arguments; we parsed them leniently so that runs could continue. On the theoretical side, the costs k, c, H, and F are illustrative, and the model treats the firm as a single decision maker. A natural extension would let the vendor of the agent choose its reliability, or let the person who takes over exert unobserved effort, turning the hand-off rule into an element of a contract. We leave these extensions to future work.

7. Conclusion

Firms that deploy autonomous agents must decide when to take tasks back. We have shown that the optimal decision rule depends on whether errors are contagious, whether tasks differ in difficulty, and whether errors derail outcomes, and that each mechanism points to a different monitoring signal. Large observational data and randomized experiments in two task domains indicate that, for the agents we study, contagion is limited while derailment is substantial and depends on what the error says. Under these conditions, a rule that hands a task back after a small, calibrated number of errors, counted by type, is not a crude approximation but the optimal policy.

References

Baird A, Maruping LM (2021) The next generation of research on IS use: A theoretical framework of delegation to and from agentic IS artifacts. MIS Quart. 45(1):315–341.

Barres V, Dong H, Ray S, Si X, Narasimhan K (2025) τ^2-Bench: Evaluating conversational agents in a dual-control environment. Preprint, submitted June 9, https://arxiv.org/abs/2506.07982.

Berente N, Gu B, Recker J, Santhanam R (2021) Managing artificial intelligence. MIS Quart. 45(3):1433–1450.

Blackwell D (1953) Equivalent comparisons of experiments. Ann. Math. Statist. 24(2):265–272.

Chow YS, Robbins H, Siegmund D (1971) Great Expectations: The Theory of Optimal Stopping (Houghton Mifflin, Boston).

Fügener A, Grahl J, Gupta A, Ketter W (2022) Cognitive challenges in human–artificial intelligence collaboration: Investigating the path toward productive delegation. Inform. Systems Res. 33(2):678–696.

Hawkes AG (1971) Spectra of some self-exciting and mutually exciting point processes. Biometrika 58(1):83–90.

Heckman JJ (1981) Heterogeneity and state dependence. Rosen S, ed. Studies in Labor Markets (University of Chicago Press, Chicago), 91–140.

Holmström B (1979) Moral hazard and observability. Bell J. Econom. 10(1):74–91.

Jovanovic B (1979) Job matching and the theory of turnover. J. Political Econom. 87(5):972–990.

Kirsch LJ (1997) Portfolios of control modes and IS project management. Inform. Systems Res. 8(3):215–239.

Nebius (2024) SWE-agent trajectories. Dataset, Hugging Face, https://huggingface.co/datasets/nebius/SWE-agent-trajectories.

Puterman ML (1994) Markov Decision Processes: Discrete Stochastic Dynamic Programming (Wiley, New York).

Shiryaev AN (1963) On optimum methods in quickest detection problems. Theory Probab. Appl. 8(1):22–46.

Wald A (1947) Sequential Analysis (Wiley, New York).

Wang RY, Strong DM (1996) Beyond accuracy: What data quality means to data consumers. J. Management Inform. Systems 12(4):5–33.

Widyasari R, Sim SQ, Lok C, Qi H, Phan J, Tay Q, Tan C, et al. (2020) BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. Proc. 28th ACM Joint Meeting European Software Engrg. Conf. Sympos. Foundations Software Engrg. (ACM, New York), 1556–1560.

Yang J, Jimenez CE, Wettig A, Lieret K, Yao S, Narasimhan K, Press O (2024) SWE-agent: Agent-computer interfaces enable automated software engineering. Adv. Neural Inform. Processing Systems 37.

Yao S, Shinn N, Razavi P, Narasimhan K (2024) τ-bench: A benchmark for tool-agent-user interaction in real-world domains. Preprint, submitted June 17, https://arxiv.org/abs/2406.12045.

Appendix A. Proofs

Proof of Proposition 1. Let T denote the operator on the right-hand side of (2), acting on bounded functions on [0, ∞). For bounded W_1 and W_2, $|TW_1(x) - TW_2(x)| \leq (1 - q)\|W_1 - W_2\|$, so T is a contraction and has a unique fixed point W with $0 \leq W \leq H$ (Puterman 1994). Suppose W is nondecreasing. For each e, $x \mapsto W(\delta x + \alpha e)$ is nondecreasing; because Poisson(μ + x) increases in x in first-order stochastic dominance, E[W(δx + αe)] is nondecreasing in x. Hence C and TW = min{H, C} are nondecreasing. The set of bounded nondecreasing functions is closed under uniform convergence, so the fixed point is nondecreasing. Because c > 0, C is strictly increasing, and {x : C(x) ≥ H} is an interval [x*, ∞). ∎

Proof of Proposition 2. If δ = 1, x_t = αN_t, and the rule x_t ≥ x* is the rule N_t ≥ x*/α. If δ < 1 and α > 0, two histories with the same count can have different intensities; for example, a single error at step t − 1 gives x_t = α, whereas a single error at step t − j with j > 1 gives αδ^{j−1} < α. When x* is interior, histories of this kind on either side of x* occur with positive probability. A count rule takes the same action on both, whereas the optimal rule does not, and because C is strictly increasing the count rule's action is strictly worse on one of them. ∎

Proof of Proposition 3. Without hand-off, E[x_{t+1}] = ρE[x_t] + αμ with x_0 = 0, so E[x_t] = αμ(1 − ρ^t)/(1 − ρ). Step t is reached with probability (1 − q)^t, so C_U = Σ_t (1 − q)^t[k + cμ + cE[x_t]], which sums to the stated expression when (1 − q)ρ < 1 and diverges otherwise. ∎

Proof of Proposition 4. Both static choices are feasible, so W(0) ≤ min{H, C_U}. If α = 0, then x_t = 0 for all t, the history carries no information, and W(0) = min{H, C_U}. Since W(0) ≤ H for all parameters and C_U diverges as (1 − q)ρ → 1 by Proposition 3, C_U − W(0) grows without bound. ∎

Proof of Proposition 5. The posterior after (t, N) is gamma with shape a + N and rate b + t, and the predictive count is negative binomial with size a + N and success probability (b + t)/(b + t + 1). This family increases in N in first-order stochastic dominance, and the expected step cost k + c(a + N)/(b + t) is strictly increasing in N. Backward induction on a horizon T, whose terminal value matters with probability at most (1 − q)^T, yields a value function that is nondecreasing in N for each t, so the stopping region at t is {N ≥ N*(t)}. As t → ∞ the posterior concentrates at μ, and the problem converges to the known-μ problem with α = 0, whose solution is to hand off if and only if (k + cμ)/q ≥ H. ∎

Proof of Proposition 6. With α = 0 and μ known, the posterior probability of derailment after N errors is π_N = 1 − (1 − θ)^N, so N is a sufficient state. Consider the one-step look-ahead comparison: stop now at cost H, or take one more step and then stop. The latter costs k + cμ + q F E[π_{N+e}] + (1 − q) H, where E[1 − π_{N+e}] = (1 − θ)^N E[(1 − θ)^e] = (1 − θ)^N e^{−μθ}. Stopping is at least as good if and only if qH ≤ k + cμ + qF[1 − (1 − θ)^N e^{−μθ}], which rearranges to (3). The right-hand side of this inequality is nondecreasing in N, and N never decreases, so once the inequality holds it holds at all later states. The problem is therefore monotone in the sense of Chow et al. (1971, Theorem 3.5), and the one-step look-ahead rule is optimal. The boundary cases follow directly from the inequality. ∎

Proof of Corollary 1. With independent derailment by type, the probability that the task is not derailed after counts (N_1, …, N_K) is Π_k (1 − θ_k)^{N_k} = exp(−Σ_k w_k N_k), and the expected factor contributed by one more step is exp(−Σ_k μ_k θ_k). The one-step look-ahead comparison of the proof of Proposition 6 then holds with these substitutions, the weighted count never decreases, and the monotone case applies as before. ∎

Proof of Proposition 7. With both channels the state is (x, N) and the Bellman operator is TW(x, N) = min{H, k + c(μ + x) + E[q F π' + (1 − q) W(δx + αe, N + e)]}, where π' = 1 − (1 − θ)^{N+e} and e ~ Poisson(μ + x). The operator is a contraction with modulus 1 − q. If W is nondecreasing in (x, N), the integrand is nondecreasing in e, x, and N, and e increases stochastically in x, so TW is nondecreasing; the fixed point inherits the property. The stopping region {C(x, N) ≥ H} is therefore an upper set, and for each x the smallest stopping count N*(x) is nonincreasing in x. ∎

Proof of Proposition 8. Every policy π has expected cost A_π + H·P_π, where A_π is the expected cost of the agent's steps and wrong outputs and P_π is the probability of hand-off; neither depends on H. The optimal cost W(0; H) = inf_π (A_π + H P_π) is therefore concave and nondecreasing in H, with W(0; 0) = 0. For H ≤ C̄, V(H) = H − W(0; H) is convex with V(0) = 0 and V ≥ 0, and a convex function that is zero at the origin and nonnegative is nondecreasing on [0, C̄]. For H ≥ C̄, V(H) = C̄ − W(0; H) is nonincreasing. Hence V is maximized at H = C̄. ∎

Appendix B. Details of the Experiments

Models and access. All agents were accessed through OpenRouter with temperature 0.5. The commercial models were run with low reasoning effort, and prompt caching was enabled for Claude Sonnet 5.5. Requests to a given model may be served by several providers. We excluded two providers for the Llama models that rejected histories containing parallel tool calls, and providers for DeepSeek-V3.1 whose tool templates failed. Llama-3.1-8B often returned tool arguments that were not valid JSON, for example two objects concatenated; in both studies we kept the first object and recorded the event, rather than ending the run. Failures caused by the provider were retried; failures caused by the agent's output were kept as outcomes.

Study 1. We used τ²-bench version 2.0 in text mode with its original airline and retail task sets, the default language-model agent, and the default user simulator, played by GPT-4.1-mini.

Success is the benchmark's environment check, which compares the final database with the database produced by the correct sequence of actions; we did not use the language-model judges. The injection intercepts the environment's response to the agent's tool calls. Injected calls are not executed, and the benchmark's evaluation replays them with the same injected response, so that injections cannot change the database. DeepSeek-V3.1 and the commercial models were run with two repetitions because of provider rate limits and cost.

Study 2. The 87 bugs come from youtube-dl (38), thefuck (32), tqdm (7), httpie (5), fastapi (3), cookiecutter (1), and tornado (1). For each bug we built the buggy version with its dependencies in a separate environment, added the developer's test files from the fixed version, and recorded the fail-to-pass and pass-to-pass tests by running them before and after applying the developer's patch. Each run starts from a clean copy of the buggy version and is reverted afterwards; runs on the same bug are serialized across agents. The agent's prompt contains the identifiers of the failing tests and the last lines of their output. Tool outputs are truncated to 4,000 characters, file views to 150 lines, and test runs to 300 seconds.

Analysis. Agent errors are tool calls that the real environment rejected. The window analysis compares the error rate over the five calls after the injection point with the same window in control runs, using runs that reached the injection point. The lag regressions pool all arms, include task fixed effects and the log of the call position, and are estimated on tasks with variation in the outcome. Intervals use 1,000 bootstrap resamples of tasks. The total cost of the 11,352 runs, including pilots, was about US$330. Code and run-level data are available in the replication package.